

A knowledge-based framework for extracting components in agile systems development

Vijayan Sugumaran · Mohan Tanniru ·
Veda C. Storey

Published online: 24 July 2007
© Springer Science+Business Media, LLC 2007

Abstract Considerable strides have been made in the use of components in software development. Many proprietary enterprise resource planning (ERP) software environments use modular components to develop and customize “best practices” to meet a specific organizational need. In agile application development, many developers and users are asked to design systems in a short period of time. These applications may use components that are embedded in software repositories. The challenge then is how to select the right software components (data and procedures) to meet an application requirement. Although experienced developers may select and customize components to meet the needs of an application, such expertise may not be available to other applications. This paper presents a knowledge-based framework to select and customize software components and demonstrates its value in deriving quality specifications, even when the developers are relatively inexperienced.

Keywords Component reuse · Domain model ·
Agile systems development · Component repository

V. Sugumaran (✉)
Department of Decision and Information Sciences,
School of Business Administration, Oakland University,
Rochester, MI 48309, USA
e-mail: sugumara@oakland.edu

M. Tanniru
School of Business Administration, Oakland University,
Rochester, MI 48309, USA
e-mail: tanniru@oakland.edu

V. C. Storey
J. Mack Robinson School of Business Administration,
Georgia State University, 60 Broad St., Atlanta, GA 30303-4015,
USA
e-mail: vstorey@gsu.edu

Facing an increasingly competitive and uncertain business environment, enterprises are seeking new strategies and technologies to implement an agile and flexible software architecture that will allow them to react quickly to market changes. Modularity in the management of software projects became fundamental in the 70's as the size of software development efforts started to increase (e.g. evolution of chief programmer teams). In the early 80's, when firms wanted flexibility in software to meet changing user's requirements and minimize software maintenance costs, the focus shifted to modularizing the code with “well defined” structures (structured programming and design). These structures are intended to make code change easier when requirements change. Similarly, when firms wanted to reuse code in large software development projects, each major function was decomposed into smaller ones. In the 90's, the reuse of software objects became more dominant as applications started to incorporate more and more domain knowledge with increased potential for a certain degree of configurability. However, their reuse is complicated because of a lack of context that is generalizable or the context embedded in the reusable component is not explicit enough to appreciate its potential reuse. In essence, as software development moved up the application chain, a greater degree of context is being introduced to enable its wider applicability, but the context embedded has a design focus (bottom-up), as opposed to a process focus (top-down).

In the 2000's, the Internet became pervasive so organizations started to make their applications transparent for external use by their business partners. As a result, agile methods have become more important for the development of flexible applications. The underlying assumption is that more and more application components (procurement,

ordering, payment, etc.) need to be easily accessible by external stakeholders and business partners across the organizational value chain to meet changing business dynamics so they must be web-enabled and transparent [1]. Therefore, if business agility is critical for competitive survival, then agile methodologies are needed to enable applications to be modularized to support such cross-organizational use. A bottom-up, value-chain driven approach to componentization can help incorporate a process or business context into these components. However, if the business domain knowledge is driven by an organizational need to compete by explicating its broader objectives and defining major processes to service these objectives, then software components formed to support internal design or external partnerships can be mapped to these business processes.

The objective of this research is to develop a framework that can relate concepts stored in a domain knowledge repository to components stored in a software component repository so that developers can use these two repositories interactively to search and retrieve relevant components during systems development. The contribution of doing so is to make this approach to software development feasible and effective, thus facilitating agility in software development.

The remainder of this paper is organized as follows. Section 1 discusses related research in the areas of agile development methodologies, domain modeling, and component-based development. Section 2 presents our methodology and discusses a knowledge-based framework for structuring domain knowledge and relating it to software component repositories. Section 3 discusses the architecture of the prototype and provides implementation details. Section 4 discusses the empirical validation of the methodology and Sect. 5 provides a discussion of the usefulness of the approach and the limitations of this research. Section 6 summarizes and concludes the paper.

1 Related research

Application agility is centered around two basic tenets: (a) being able to respond to changes in requirements through traceability of components, and (b) identifying potential reusable components that can be incorporated into a new system created through refactoring [2]. Component based development addresses similar problems. Domain knowledge is a key ingredient in supporting these two tenants [3]. This research combines the advances in domain modeling and component retrieval in order to support agile systems development.

Agile software development methods are characterized as incremental, cooperative, straightforward and adaptive

[4]. They are incremental in the sense that small software releases are inherent in the methodology, as are rapid development cycles. These methods promote a close interaction between developers and customers. They are also easy to understand, modify and implement because they are well documented. The most important characteristic, however, is the adaptive and responsive nature of the approach to changes in requirements [5]. The success of an agile methodology depends on its ability to: (1) identify the impact of changes in the user's requirements in terms of the components to be modified; and (2) being able to support and manage the change process [3]. One way to facilitate this process is to clearly establish explicit relationships between business objectives and processes and the individual components that implement these processes. This provides a traceability mechanism between requirements and components that implement them.

Agile methodologies rely on "Refactoring" to create reusable assets that can be utilized in systems development. Refactoring is the process of "cleaning up" artifacts to make them more reusable [6]. For example, chunks of source code can be modified to improve internal consistency and clarity without changing the external behavior and create reusable components. Thus, the result of refactoring could be a repository of components that can be used in systems development. One issue in this scenario of rapid systems development is the ability to identify appropriate components from the repository of artifacts created through refactoring [7]. This same issue is also encountered in component-based software development.

If done correctly, "refactored assets" can provide a number of advantages. An information system developed from reusable components should: be more reliable [8], increase developer productivity [9–11], reduce skills requirement [12], shorten development life cycle [13, 14], and decrease development costs [15, 16]. Ideally, developers search for components from repositories by specifying the functionalities that meet the requirements of the system under development. However, there is no systematic support for narrowing the search space and retrieving relevant components. Developers may specify search parameters such as the programming language, component type or technology, application domain, industry type, and operating system [17]. Although these are essential, the most important aspect of searching is being able to specify requirements in a natural form.

A priori knowledge of the application domain, independent of how the software components repositories have evolved, may prove to be more useful across a wide variety of applications. For example, knowledge about an application domain, represented by key objectives, processes, actions and objects, can help a developer first identify user requirements, void of any design bias. The developer can then use

appropriate search criteria (component names, actions supported, etc.) to limit the search space and retrieve the needed software components efficiently. Existing methods rely on such domain knowledge to support the search of software components.

1.1 Agile development methodologies

Agile software development involves an iterative approach where new functionalities are added in an incremental fashion [18]. In contrast to traditional system development methodologies, agile methods are “light-weight methods” that focus on customer satisfaction through rapid delivery of useful software and accommodate changes in requirements [19]. They minimize resource-intensive intermediate artifacts and use small development teams. Some of the well-known agile development methods are: Extreme Programming (XP), Adaptive Software Development (ASD), Dynamic Systems Development Method (DSDM), Feature Driven Development (FDD), and Scrum [20].

Extreme Programming emphasizes rapid and frequent feedback to the customers and end users, unit testing and continuous code reviews [21]. It also relies on continuous refactoring, collective code ownership, and pair programming [22]. One limitation of this approach is that it does not provide any formal support for refactoring in terms of component identification and customization [23]. ASD is suited for the development of large and complex systems and encourages incremental and iterative development [24]. It is feature based, iterative, risk driven, and change tolerant [25]. However, it does not provide adequate support for project management. The Dynamic Systems Development Method attempts to alleviate common problems associated with systems development such as exceeding budgets, missing deadlines, lack of user involvement and top management commitment [26, 27]. Although it uses rapid application development philosophy, it does not provide enough support for component reuse.

Feature Driven Development (FDD) is a model-driven approach, comprised of five different activities: (a) develop overall model, (b) build feature list, (c) plan by feature, (d) design by feature, and (e) build by feature [28, 29]. While FDD is built around a core set of industry-recognized best practices, it does not provide adequate support for component traceability and reuse. The Scrum methodology allows developers to choose appropriate development techniques and does not prescribe a sequence in which the activities must be implemented [30]. Developers can start with any activity, and change between activities at any time, thus, increasing the project’s flexibility and productivity [31]. While Scrum supports flexible schedules and deliverables and small teams,

it requires hand-on management with a high degree of coordination.

1.2 Domain modeling

Domain analysis research has explored many ways to identify various components of business domain knowledge at a high level of abstraction (e.g. objectives, processes and actions) and to relate them to organizational processes [32, 33]. Tracz [34] proposes the inclusion of a lexicon (meaning of terms), ontology (set of elements to describe a domain), taxonomy of elements, relationships, products, actions and events. Others focus on specific domain level artifacts and associated information for reuse, including processes (specifications, sequence and structure), data (attributes, relationships and structure), and entities (events, agents, and objects) [35–37].

Several methods have been proposed to structure this domain knowledge, once identified [38, 39]. The Rockwell-Collins organization uses a synthesis approach to move from a particular domain to application, using concepts such as terms, assumptions, and decision models [40]. The Software Engineering Institute’s feature-oriented domain analysis (FODA) includes context analysis, domain modeling, and architecture modeling [41, 42]. In IdeA, design schemas map problems to solutions [43]. In the Triadic domain model-based software development method, the domain problem model (including solution methods, algorithms, data structures, etc.) is linked to a domain process model that identifies typical activities and constraints [44–46].

1.3 Component based development

Component based development (CBD) is changing the way applications are being developed and delivered to the end user [47–50]. It requires a shift in software development approaches, particularly with the emergence of component architecture standards such as CORBA, COM, and EJB [51–53]. The roles of software components and component models to improve productivity have been analyzed previously [8, 54, 55]. The focus of software development is shifting from automatic code generation to combining high-level software components to meet user requirements. Tools and techniques have been introduced to provide such support [17, 56, 57]. An information system developed from reusable components is more reliable [8], increases developer productivity [9–11], reduces skills requirement [12], shortens development life cycle [13, 14], and decreases development costs [15, 16].

Ideally, developers search for components from repositories by specifying the functionalities that meet the

requirements of the system under development. However, there is no systematic support for narrowing the search space and retrieving relevant components. Commercial component repositories allow developers to specify search parameters such as the programming language, component type or technology, application domain, industry type, and operating system [17]. As more and more of these components become available as part of a vendor's software environment for application development, the issue becomes one of selecting the appropriate components to build an application [58].

Prior research has examined various methods for search and retrieval of software components that rely on informational content [59, 60], operational semantics [61, 62], denotational semantics [63, 64], topologies [65, 66] and structures [67, 68]. UML 2.0 allows for more appropriate descriptions of a component's dependencies and its interface specification so that components can be retrieved from repositories based on functional or behavioral criteria [69]. Several formal methods based approaches have been identified for matching components to user needs [70–73]. For example, Chen and Cheng [74] discuss a semantic foundation for specification matching between software requirements and components. They define a component interface generality relation that provides a foundation for component selection [75]. Formal specification is used to represent the generality relationship in terms of the semantic obligations of the component to satisfy the requirement. They present specification matching as a method for classifying, retrieving and modifying components [71]. Similarly, Penix and Alexander [76] use formal specifications of components to relate to domain specific

feature-sets. This linkage is used to improve component identification and retrieval [72]. However, this approach depends on formulating the feature sets for each domain.

Factors such as the size of the repository, query representation, asset representation, storage structure, navigation scheme, relevance, and matching criteria impact the search and retrieval process [77, 78]. While tagging a software component with knowledge or structure might help its use in a given application environment, it does not provide a general solution. In fact, most of the retrieval methods discussed in the literature are suitable for a particular domain, but are not tailored to provide an effective overall retrieval solution. In addition, existing retrieval methods do not relate components to business objectives and processes, which change quite frequently in this global, Internet age. Thus, linking components to business processes enable organizations to better respond to changes in the environment by quickly adapting existing applications. Our proposed approach facilitates this in a systematic way.

2 Proposed methodology

2.1 A framework for structuring domain knowledge

The variability in existing approaches reflects the differences in the goals of an organization for capturing domain knowledge. A single unified approach may not be feasible or even desirable [79]. Sugumaran et al. [80] organize domain knowledge from business objectives to higher-level data objects via processes and actions as shown in Fig. 1. The domain knowledge includes processes (specifications,

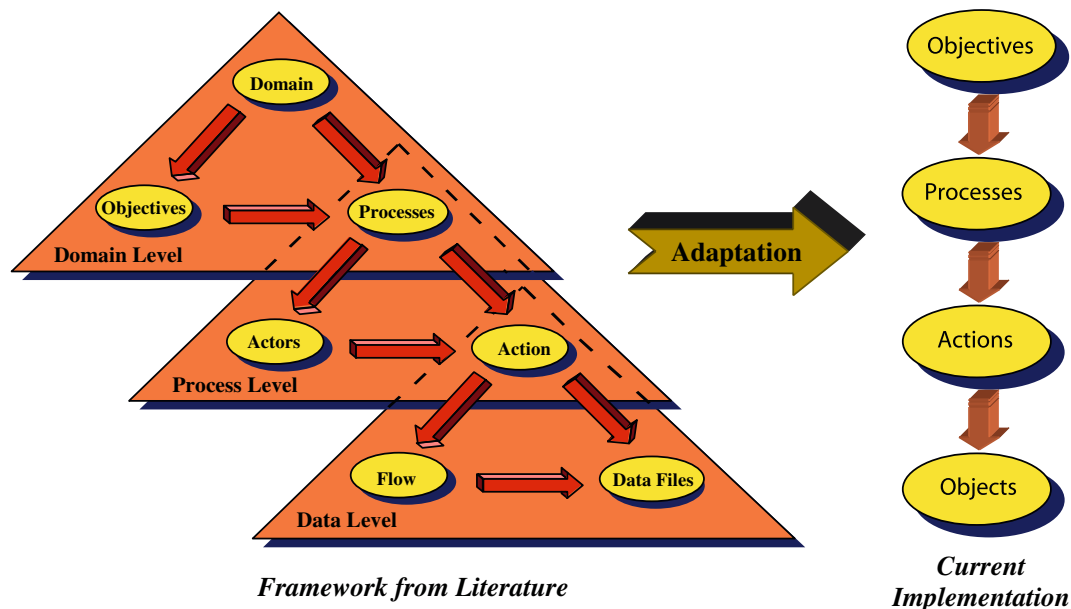


Fig. 1 A structure for capturing domain knowledge

sequence, structure), data (attributes, relationships and structure), and entities (events, agents, objects), all capturing information traditionally gathered during the analysis phase (except for certain macro level business

objectives). This structure is used in this paper as the basis for software component selection.

The structure in Fig. 1 is used for storing domain knowledge used during the analysis phase of information

Objective	Processes	Actions	Objects
Define potential market segments	Determine product share Analyze competition Analyze economic conditions Create market segment ⋮	Determine total demand for product Determine market share Analyze competitor sales discounts Analyze competitor advertising Determine GDP growth Determine inflation rate Design proper packaging for segments Develop advertising strategies ⋮	Product Competitor Market Indicators ⋮
Analyze Sales Prospects	Identify potential prospects Sell to prospects Analyze prospect feedback ⋮	Obtain mailing lists Gather customer preferences Generate short list of prospects Make and follow-up on appointments Show product lines Follow-up prospects List of successful and failed prospects Generate reasons for failure Generate profile of successful prospects ⋮	Prospects Products Prospect Survey ⋮
Bid on large customer requests	Obtain RFPs Evaluate bid strategies Prepare and submit bids Analyze bid response ⋮	Gather information about RFPs Pricing analysis Timing analysis Evaluate logistics Clarification on bids Prepare bid in proper format Bid submission Analyze reasons for success and failure Generate Lessons learned Identify things to improve ⋮	RFP Bid Customer Product ⋮
Make Sales	Process_order information Process_return information ⋮	Enter sales order Follow-up orders Cancel sales order Hold sales order Close sales order Analyze reasons for return Analyze product defects ⋮	Sales Person Order Customer Product Returns Invoice ⋮
Follow-up on Sales	Maintain customer information Analyze customer feedback ⋮	Record customer details Obtain customer feedback Analyze product returns Analyze customer service Provide feedback to manufacturing ⋮	Customer Returns Customer survey ⋮
Analyze sales performance	Analyze customers Analyze sales cycle Analyze performance ⋮	Develop customer profile and demographics Analyze customer buying habits Analyze frequency of purchase Analyze seasonality of products Analyze profits and productivity Analyze cost Analyze use of technology Assess commission to sales force ⋮	Customer Service object Product Invoice Sales Person ⋮

Fig. 2 Partial sales domain model

systems development [80, 81]. Also referred to as a domain model, it has three levels: *domain*, *process* and *data*. The domain is defined as an area affected by a business strategy. It can be as broad as an entire organization (e.g. the sales strategy is to improve a firm's competitiveness), or as narrow as the needs of a functional area (e.g. analyze sales performance to identify which customers are buying and what they are buying). This level provides relatively context-free information about the domain, by focusing on basic domain objectives, major business processes, and other non-process-specific information (industry categorization, performance measures, etc.).

The process level corresponds to the way a firm operationalizes its business strategy. This level provides context-specific information such as who performs these processes, where they are performed, and what sub-processes or activities are associated with these processes. One can capture the rationale associated with these processes and any relevant assumptions or constraints. The data level examines the data objects, such as files and events, to support these processes through various actions. This structure is depicted using three inter-locking triangles, with *higher-level processes* linking the domain and business process levels, and *lower-level actions* linking process and data levels. By keeping these three separate but connected, the domain level can continue to reflect changes in business strategies whereas the data level can continue to reflect changes in the technology implementation.

An example of a sales domain, extracted from various experts involved in the software development of order processing systems of a major software vendor, is presented in Fig. 2. Six sales domain objectives are related to several business processes and actions. Although there is no overlap among the actions (i.e. the same action is used by many processes), the data objects associated with each business process has an overlapping impact on other business processes.

2.2 Methodology steps

The methodology consists of three major steps: (1) knowledge based retrieval of software components, (2) consistency checking of selected components, and (3) customization of components, each of which is described below.

2.2.1 Retrieval of software components using the knowledge-based framework

The focus of this work is to assist the developer in retrieving the relevant components from the repository, based on the requirements defined within the context of the domain knowledge. For example, if the developer selects a

process called "Process_order information," it can lead to the identification of data objects such as "order" and "customer," along with actions such as "enter sales order," "follow_up orders," and "close sales order". The detailed objects/methods chosen must support the implementation of, for example, the process: *process_order information*. Reusable repositories already provide various attributes such as "order date, customer name, order value, etc." for the *Order* object. Some of these attributes and any associated methods may be reused in the design of a new system (see Fig. 3).

All attributes considered necessary to support various actions identified during the analysis phase are chosen first. The access and update methods for all the selected attributes are automatically included in the target system. Other methods unique to the higher level objects (which may not have been identified in the analysis phase) are presented to the developer for review and possible inclusion. These are referred to as "actual" or "implementable" objects. Figure 3 illustrates the Customer, Product and Order data objects (or tables), along with various methods associated with them. The developer may identify some of the attributes and methods needed to support the actions identified in the domain model. Based on the attribute and method selections, the actual objects are configured by eliminating unnecessary attributes and methods and "re-packaging" these objects.

2.2.2 Consistency checking of software components chosen

As the developers use the domain knowledge to select components, consistency checks must be performed before the components can be integrated. In many programming environments, methods call other methods. Furthermore, some attributes stored in the repository may need to be customized to reflect the conventions used in the specific problem domain. In the prototype developed as part of this research (Sect. 3), many of the consistency checks are performed automatically. The types of consistency checking are: (1) Objective–Process Consistency, (2) Object–Attribute Consistency, and (3) Method–Action Consistency.

The first two types are intra-repository checks. They ensure that the components extracted from either the domain or object repositories are *internally consistent*. For example, a user may select a set of objectives and then a set of processes to meet these objectives. It is critical that a selection does not violate internal process dependencies (one process needs another process and, in turn, meets an objective). Similarly, if attributes of an object are selected, the system must ensure that the user is aware of other attributes that are needed since the associated methods that manipulate these attributes need them. For example,

Customer Object		
Attributes	Methods	JavaBeans Implementation
Name Billing address Credit limit Credit history Customer type Contact information Average sales volume Last transaction date Discount status	Access methods Update methods Review customer credit Review discount status . .	//Customer Class - with Bound property import java.beans.*; import java.util.*; public class Customer extends Object { protected String name; //instance variables protected String address; protected float credit_limit; ... //additional instance variables //constructor
Product Object		
Attributes	Methods	JavaBeans Implementation
Product Number Product Name Description Technical specification Warehouse location Qty in stock Qty on order Qty back ordered Reorder point Reorder quantity List price Bid price Standard cost Product line	Access methods Update methods Review bid price Review reorder point Review safety stock Review reorder quantity Analyze back orders . .	//Customer Product - with Bound property import java.beans.*; import java.util.*; public class Product extends Object { //instance variables protected int product_number; protected String product_name; protected String description; ... //additional instance variables PropertyChangeSupport changeSupport; //constructor public Product() { changeSupport = new PropertyChangeSupport(this); } //Property Change Listener Support Methods public synchronized void addPropertyChangeListener (Prope
Order Object		
Attributes	Methods	JavaBeans Implementation
Order No. Order date Qty ordered Qty backordered Expected delivery date Qty not filled Order value Order status Order credit status Customer name Sales person name Shipping address Distribution location Special promotion details Delivery dates	Access methods Update methods Analyze lost sales Analyze promotional cost effectiveness Analyze order cycle time Compute order credit status . .	//Customer Order - with Bound property import java.beans.*; import java.util.*; public class Order extends Object { protected int order_number; //instance variables ... //additional instance variables PropertyChangeSupport changeSupport; //constructor public Order() { changeSupport = new PropertyChangeSupport(this); } // ** Property Change Listener Support Methods ** // bean properties public void setOrder_number (int new_ord_num) { order_number = new_ord_num; } public int getOrder_number () { return order_number; } public analyze_lost_sales () { int temp = this.getQtynotfilled();

Fig. 3 Sample objects from object repository

selection of *<order, order credit status>* may require *<customer, credit limit>* because the method that evaluates *<order credit status>* requires *<credit limit>*. Of course, as a part of the customization, a developer may remove some of these dependencies by altering the method.

The third type of consistency is an inter-repository check. Since the knowledge is extracted from repositories that are developed independently, it is critical that the

developer check for these separately. For example, if a method is chosen from the component repository, it should correspond to some action chosen in the domain repository and vice versa. Whereas the first two types of consistency are implemented automatically using rules stored in the knowledge base, the third type of consistency is left up to the developer and could become part of the customization or repository revision.

2.2.3 Customization of components

When appropriate, the selected components may be customized to meet specific user and/or process requirements. This is because no system can be expected to be fully supported without some type of customization, unless the organization makes a conscious decision to redesign its processes to fit the “best practices” embedded in the domain model. The customization might involve changing the objects chosen to reflect the terminology, needs, standards, and business constraints of a firm. For example, a firm may want to refer to the attribute “product number” as “item#” or “product name” to “product label”. The developer may modify the pre- and post-conditions of different methods, as well as customize one or more methods to reflect the standard operating procedures of a firm. For example, the “*analyze lost sales*” method of the ORDER object could be triggered at different times by modifying the pre-conditions. This method may also be re-implemented to reflect the techniques/heuristics that the firm uses to analyze lost sales, depending upon safety stock levels. The developer may add new methods to the object, but is responsible for ensuring that the new methods are consistent with the remaining attributes and methods. Thus, the developer may perform one or more of the following sub-steps: (1) modify attribute and method names, (2) modify pre and post conditions of methods, (3) modify the

implementation of methods, and (4) add new methods. Figure 4 summarizes the steps.

3 Prototype implementation

The Systems Analysis and Design Assistant (SADA) provides suggestions to the developer by reusing higher-level domain knowledge and lower-level data object artifacts. A developer can interact with SADA to select an appropriate set of concepts from the system’s domain model and to provide additional information for resolving conflicts. The architecture of SADA system is shown in Fig. 5. The system comprises of: (a) User Interface Module, (b) Knowledge-Base Module, (c) Local Data Module, and (d) External Repositories. SADA has been implemented using Jess, an expert system shell written in Java [82]. Jess was chosen for several reasons. First, the methodology could be expressed as a set of rules, which can easily be translated into Jess. Second, since Jess is written in Java, a web-based version of the system can easily be created. Third, the architecture of Jess is flexible and extendable, so user-defined functions are easily integrated. Finally, the object repository uses the Java Beans component architecture. Jess has the capability to interface with this repository to retrieve and customize objects based upon user requirements.

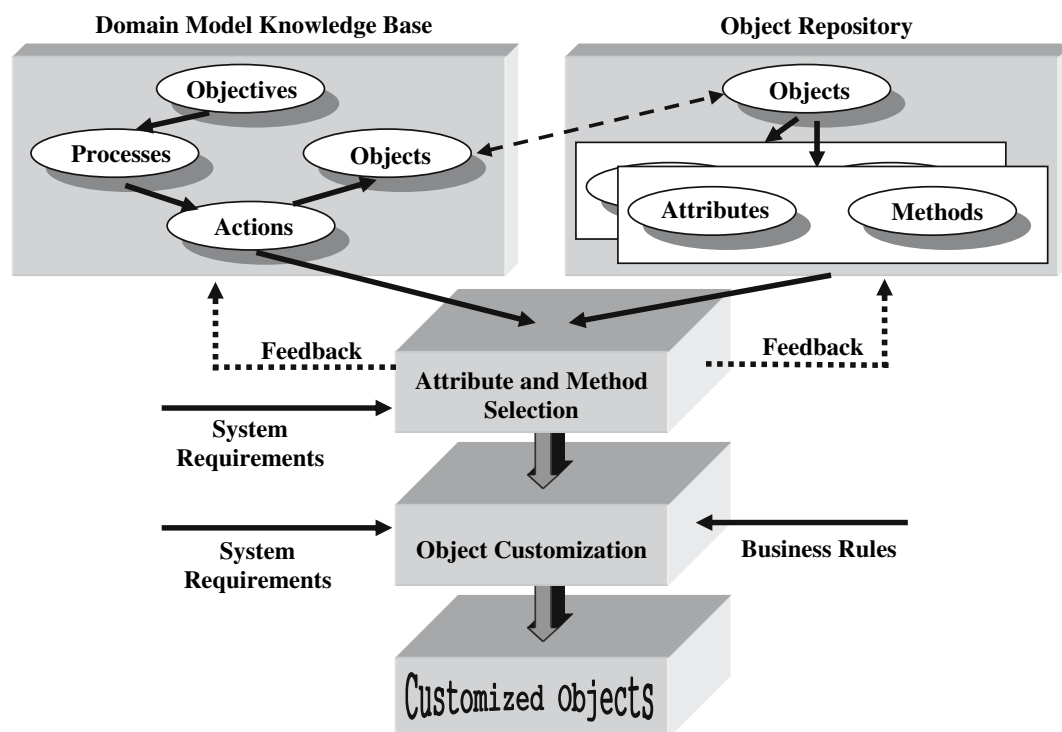
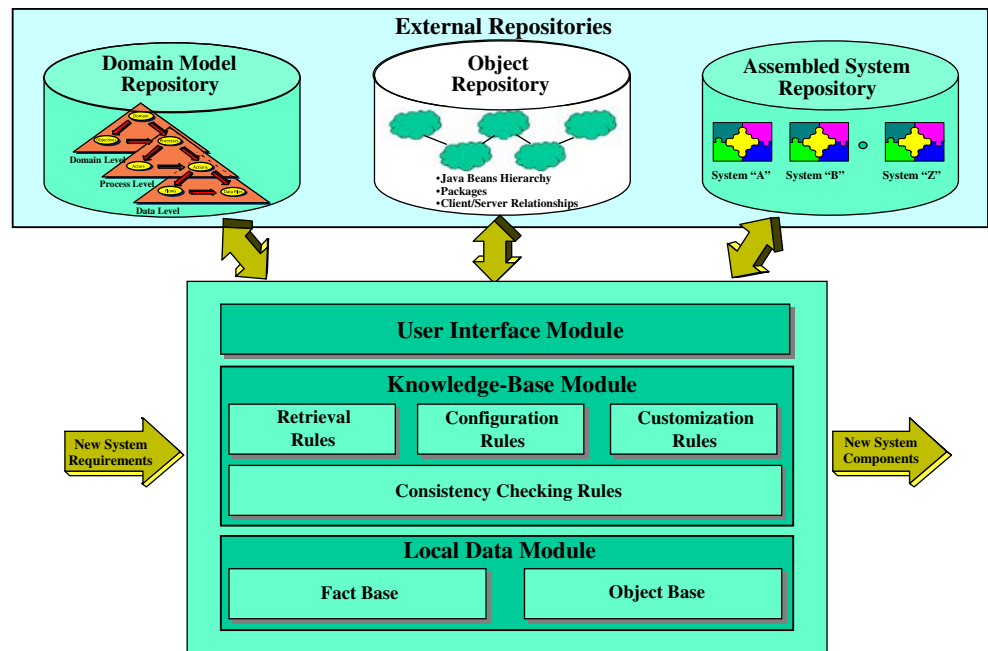


Fig. 4 Methodology for reuse in analysis and design stages

Fig. 5 Architecture of systems analysis and design assistant (SADA)



3.1 User interface module

The user interacts with SADA when specifying the requirements for a new system using a standard interface. The user interface module provides mechanisms for the developer to browse domain models and learn about a particular application domain as well as to explore the object repository and templates of previously assembled systems. The developer can specify the requirements for a new system in terms of desired functionalities, which are mapped to the domain model and an appropriate set of objectives, processes, actions and objects are identified. When specifying requirements for a new system, the user is first shown a set of high level objectives from which a few are selected as shown in Fig. 6 (Objectives Elicitation Panel). Once the user selects the objectives, the system provides the high level processes associated with the

chosen objectives. The user can delete some of the processes that may not be needed. While selecting artifacts, the system performs several types of consistency checking and elicits additional input from the user to resolve conflicts. For example, if the user attempts to delete the “Sell to Prospects” process from the new system, SADA prevents this because the “Analyze Prospect Feedback” process requires it, and an alert message is displayed, as shown in Fig. 7.

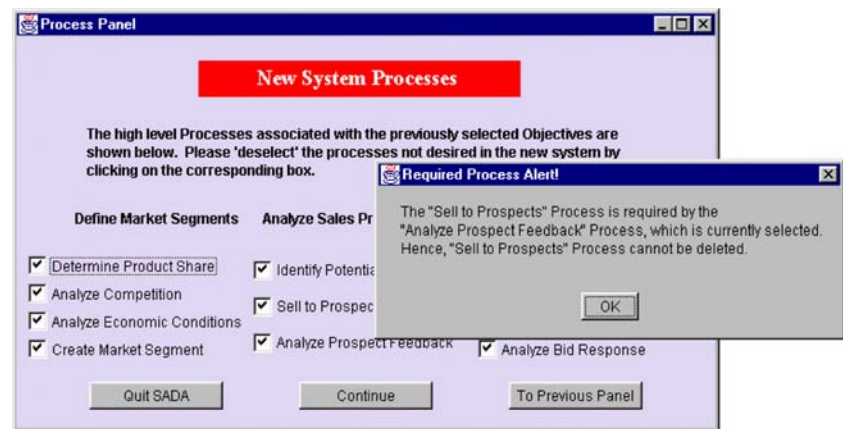
3.2 Knowledge-base module

The knowledge-base module contains procedural and control knowledge required to retrieve artifacts from the repositories and to assemble and configure the user-selected components. Four categories of rules provide the basis for the intelligence of the system.

Fig. 6 SADA initial and objectives panel



Fig. 7 Dependency checking during process selection/deletion



3.2.1 Retrieval rules

Retrieval rules enable SADA to locate and retrieve domain model artifacts from the domain model repository and transform them into a format that is usable by the system. When the user selects a set of actions to be included in the new system, the retrieval rules identify the necessary components and retrieve them from the component repository.

3.2.2 Configuration rules

The generic components retrieved from the component repository may contain attributes and methods not needed in the new system. The configuration rules allow the user to “configure” the components by deleting the unwanted attributes and methods. In the sales domain, if the organization does not allow back orders, the “order” object does not need the “quantity back ordered” attribute and its related methods. The user can configure the system that has been assembled in the current session or chose another system assembled in a previous session. Before starting the configuration process, SADA presents the objects that have been included in that system, so the user can configure one or more of them.

When the user wishes to customize an object through its attributes, SADA presents all the attributes so the user can remove those not needed. The user can also receive a brief explanation about the purpose and use of an attribute. After deleting the superfluous attributes, the user can configure the methods.

3.2.3 Customization rules

Customization rules allow the user to add and/or redefine attributes and methods to meet the requirements of the new system. For example, the user may add additional attributes such as “order type” and “customer standing” or re-specify how back orders are processed. Although

SADA provides basic consistency checking, the user must ensure the changes are consistent. The user customizes an object by starting with its attributes and then its methods. When the user wishes to customize an object through its attributes, SADA presents all the attributes so the user can identify the candidate attributes to be redefined or add additional attributes. Similarly, the methods are presented to the user from which the user can modify one or more methods to suite the needs of the application.

3.2.4 Consistency checking rules

Consistency checking rules are most important because they ensure the correctness of the design. These were developed to strengthen the intelligence of the system, while minimizing the effort required on the part of the user when generating a preliminary design. They do so by checking for inconsistencies and dependencies within attributes, within methods, and across attributes and methods. For example, if method “X” requires method “Y,” while selecting method “X” for the new system, SADA will automatically select method “Y” if it is not already selected to satisfy the dependency relationship. A sample consistency checking rule, written in Jess, is shown in Fig. 8.

SADA performs both intra-object and inter-object consistency checking. Intra-object consistency checking involves enforcing the attribute-to-attribute, method-to-method, method-to-attribute and attribute-to-method dependencies within an object. In other words, SADA ensures that all the dependent attributes and methods within that object are included in the new system. The inter-object consistency checking enforces the dependencies between objects. For example, a method in one object may require a method in another, and must be included in the new system if currently not selected. During consistency checking, SADA informs the user of the start and

```

(defrule single-dependency-present ""
  ?rem1 <- (to-be-selected $X Method)
  ?rem2 <- (not-selected $X Method ?p-id1)
  (Method-Dependency: $X Method ?p-id1 requires $Y Method ?p-id2)
  ?rem3 <- (not-selected $Y Method ?p-id2)
  =>
  (bind ?str1 (str-cat "Since the \" (implode$ $X) "\" Method requires \" \" (implode$ $Y) "\" Method, "))
  (bind ?str2 (str-cat "it will be automatically included in the Target System. "))
  (bind ?str3 "Click on 'OK' to include, or 'Cancel' to abort.")
  (bind ?str (str-cat ?str1 ?str2 ?str3))
  (bind ?d* (new YNDialog ?*current-pnl* "Method Deletion" ?str " OK " " Cancel ")))

```

Fig. 8 Consistency checking rule in Jess

completion of each type of consistency checking and generates a report.

3.3 Local data module

The local data module tracks the status of the development of the system and stores the artifacts selected by the user. It consists of a “fact base” and an “object base.” The fact base contains the internal representations of domain model artifacts such as objectives, processes, actions, objects, and dependencies, expressed as facts. A generic set of templates for facts correspond to the various domain artifacts. For example, the method dependencies are expressed as facts in the following format:

Format: (*Method-Dependency*: method-name1 Method m-id1 *requires* method-name2 Method m-id2)

Example: (*Method-Dependency*: Analyze Bid Response Method 11 *requires* Submit Bids Method 10)

Similarly, the pre-conditions and post-conditions of methods are expressed as facts using the following format. Pre- and post conditions are used to express constraints on the method execution and behavior. They can be checked in conjunction with execution of components to detect hazardous states and check the range of input and output, used in the specification of the components.

Format: (*Method*: method-name1 Method m-id1 *pre-condition* condition-expression Constraint c-id1)

Example: (*Method*: Analyze lost Sales Method 15 *pre-condition* “end-of-period is true” Constraint 7)

Format: (*Method*: method-name2 Method m-id2 *post-condition* condition-expression Constraint c-id2)

Example: (*Method*: Review Reorder Point Method 19 *post-condition* “reorder point>0” Constraint 8)

The object base stores details about components stored in the component repository. For each component, it contains information about the package of which it is a part, its level in the hierarchy, and related components. Sample order object information is shown in Fig. 9.

3.4 External repositories

The *domain model repository* contains reusable artifacts such as objectives, processes, actions and data objects for specific application domains that are structured using the framework shown in Fig. 1. The domain analyst can work independently to develop the domain model. The *object repository* contains a specific implementation of objects found in the domain model. In SADA, objects have been implemented using the Java Beans architecture. Parts of the customer, product, and order objects from the sales domain are shown in Fig. 4. The *assembled system repository* stores the designs generated during each session by name and domain, thus building up over time. The above-mentioned repositories are external to SADA and can be

Format:	Example:
(<i>Component</i> : component-name component-id)	(<i>Component</i> : Order Component 7)
(<i>Component-Dependency</i> : comp-name1 comp-id1 <i>requires</i> comp-name2 comp-id2)	(<i>Component-Dependency</i> : Order Component 7 <i>requires</i> Customer Component 4)
(<i>Component-Part-Of</i> : comp-name1 comp-id1 comp-name2 comp-id2)	(<i>Component-Part-Of</i> : Order Component 7 <i>contained-in</i> Order Processing Component 1)
(<i>Component-Hierarchy</i> : comp-name1 comp-id1 <i>super-type</i> : comp-name2 comp-id2 <i>sub-type</i> : comp-name3 comp-id3)	(<i>Component-Hierarchy</i> : Order Component 7 <i>super-type</i> : Transaction Component 3 <i>sub-type</i> : Partial Order 11)

Fig. 9 Sample templates for domain artifacts

developed independently and interfaced with systems other than SADA.

After assembling the new system, SADA generates a final assembly report. This report contains the objectives, processes, actions and objects included in the new system. It highlights those processes, actions, objects, attributes and methods that were automatically included to make the new system consistent as well as the new attributes and methods that were added during customization. The user can store this system configuration in the assembled system repository for future use.

4 Empirical validation

The effectiveness of the framework discussed in Sect. 2, can be measured at different levels of granularity such as component identification (step 1), internal consistency of these components (step 2), customized solution's appropriateness (step 3), and beyond (such as the time taken to generate the initial design and maintainability of the resulting system architecture). Given the narrow scope of the prototype (SADA) designed here, the validation focuses on component identification and some degree of consistency between these components.

4.1 Performance variables

The quality of the outcome, generated by the two groups: control and experimental (those that do not have access to the framework and those that do), is *not* a single measure but a *composite* measure of various items that are often extracted by the developer during the analysis and design phases. This outcome measure includes a total of seven scores. Five of these scores (H1a–H1e) correspond to objectives, processes, actions, higher-level objects, and software components such as detailed objects (with appropriate attributes and methods). The sixth score (H1f) reflects the overall consistency among the five domain and software object components chosen. A last score (H1) is generated by adding the values of all the six scores. The usefulness of the system was assessed by measuring the quality differences among these seven scores between the control and experimental groups. The following seven hypotheses were tested.

H1: Developers using SADA will produce better overall designs compared to the designs generated by developers without the use of the system.

H1a: Developers using SADA will perform better in identifying appropriate Objectives for the new system design than the developers without access to SADA.

H1b: Developers using SADA will perform better in identifying appropriate Processes for the new system design than the developers without access to SADA.

H1c: Developers using SADA will perform better in identifying appropriate Actions for the new system design than the developers without access to SADA.

H1d: Developers using SADA will perform better in identifying appropriate (higher level) Objects for the new system design than the developers without access to SADA.

H1e: Developers using SADA will perform better in identifying appropriate Detailed Objects for the new system design than the developers without access to SADA.

H1f: Developers using SADA will provide solution that is consistent to those that do not have access to SADA.

4.2 Experiment

The subjects for this study were business students (MIS majors taking database and systems analysis courses) who had no prior knowledge of the study and/or its objectives. They participated in the study to test the system's usefulness and were given optional points for their participation, but the points they received were contingent on the quality of their effort (i.e. mere attendance did not guarantee full credit).

Thirty-two subjects participated in the experiment. The subjects were given a three-hour training on the domain and object repository reuse for systems development. The first half of the training introduced them to the framework and its potential for reuse. The subjects were allowed to use the system to solve a banking problem during the second half. Although the subjects were all business students, they had varied experience in the systems area. They were asked to complete a questionnaire listing their prior background (course work, part-time work, etc.). The subjects were then classified into a relatively novice pool and somewhat experienced pool, with members from these two pools randomly assigned to two groups, A and B (control and experimental), to limit any experiential bias.

Two cases (Sales and Human Resource-HR) were used in the study. The sales domain knowledge was extracted from systems developers, who have worked in this domain for several years. The human resource knowledge base was developed using the experience of one of the researchers and two human resource managers. The reliability of the knowledge base or its comprehensiveness was not an issue in this validation, even though every effort was made to make the situation as realistic as possible. Neither of the two evaluators who scored the outcome was familiar

with these knowledge bases. The Banking, Sales and HR domain models, as well as the two test cases are available from the researchers.

Group A was given the Sales case for “analysis with the system”, and the HR case for “analysis without the system”. Similarly, group B was given the HR case for “analysis with the system” and the Sales case for “analysis without the system.” Both groups were asked to analyze their respective case with the system initially, so they were all equally familiar with the solution structure they needed to use to document the outcome. After they turned in their answers for the first case, they were given a second case to solve without the system. The data was coded in a common form, numbered for reference, randomized, and sent to two evaluators, who were not involved in the experiment. A “correct” solution was developed for both the Sales and HR cases by the researchers. Multiple correct solutions were allowed provided they were deemed relevant. The answer keys were developed in general terms to ensure that the evaluators were not pre-disposed to grading the answers that came from SADA more favorably than the answers that came from subjects who had not used SADA.

The problem of achieving consensus and objectivity in qualitative scoring is always an issue [83]. Since variability in scoring could come from the case used, the grader of the case, and the system, a two-tailed ANOVA was performed to isolate the significance of each factor that contributed to the difference in the scoring, as well as the role of the interaction among these three factors. As seen in Table 1,

the only factor that contributed to significant variance in scoring was the ‘system’, as expected. The role of the case used by subjects was tested to assess if it had any significant influence on the overall outcome measure as well as various constituent elements that make up this score. As seen in Table 2, only the presence of the system is shown to have a significant impact on the individual component and the total scores.

4.3 Results from the experiment

Table 3 provides the average scores under each scenario (with and without the system) for each individual factor that makes up the total score, and the total score itself. These were tested for their significance at the 0.01 level and the results shown in Table 3. The data supports hypotheses H1 and H1a–H1d and H1f. H1e is not supported. Thus, all facets of the score, except the detailed object definitions, were supported, implying that the presence of the system for component selection impacted the overall outcome quality as well as individual identification of objectives, processes, actions, higher level objects and overall consistency. The detailed object definition was not tested statistically as sufficient data was not available from both groups. Either insufficient time given to analyze the case or insufficient training provided to help the students understand this design process may have contributed to this lack of sufficient data. Either way, this needs to be addressed in future research.

Table 1 Results of ANOVA for overall total score

General linear model: total score versus case, system, grader						
Factor	Type	Levels	Values			
Case	Fixed	2	HR	Sales		
System	Fixed	2	With	Without		
Grader	Fixed	2	A	B		
Analysis of variance for total score, using adjusted SS for tests						
Source	DF	Seq SS	Adj SS	Adj MS	<i>F</i>	<i>p</i>
Case	1	0.5	0.5	0.5	0.02	0.899
System	1	7230.0	7230.0	7230.0	236.24	0.000
Grader	1	36.1	36.1	36.1	1.18	0.279
Case * System	1	66.1	66.1	66.1	2.16	0.144
Case * Grader	1	52.5	52.5	52.5	1.72	0.193
System * Grader	1	8.0	8.0	8.0	0.26	0.610
Case * System * Grader	1	16.5	16.5	16.5	0.54	0.464
Error	120	3672.6	3672.6	30.6		
Total	127	11082.5				

Table 2 Results of two-way ANOVA for the different scores

Source	DF	SS	MS	<i>F</i>	<i>p</i>
a. Analysis of variance for total score					
Case	1	0.5	0.5	0.02	0.898
System	1	7230.0	7230.0	236.81	0.000
Interaction	1	66.1	66.1	2.17	0.144
Error	124	3785.8	30.5		
Total	127	11082.5			
b. Analysis of variance for objectives score					
Case	1	0.07	0.07	0.01	0.909
System	1	775.20	775.20	144.28	0.000
Interaction	1	13.13	13.13	2.44	0.120
Error	124	666.22	5.37		
Total	127	1454.62			
c. Analysis of variance for processes score					
Case	1	0.13	0.13	0.07	0.793
System	1	215.28	215.28	119.14	0.000
Interaction	1	2.00	2.00	1.11	0.295
Error	124	224.06	1.81		
Total	127	441.47			
d. Analysis of variance for actions score					
Case	1	0.78	0.78	0.75	0.388
System	1	116.28	116.28	111.88	0.000
Interaction	1	0.03	0.03	0.03	0.863
Error	124	128.88	1.04		
Total	127	245.97			
e. Analysis of variance for objects score					
Case	1	1.13	1.13	0.94	0.335
System	1	166.53	166.53	138.88	0.000
Interaction	1	1.13	1.13	0.94	0.335
Error	124	148.69	1.20		
Total	127	317.47			
f. Analysis of variance for consistency score					
Case	1	0.28	0.28	0.15	0.701
System	1	357.78	357.78	188.49	0.000
Interaction	1	3.78	3.78	1.99	0.161
Error	124	235.38	1.90		
Total	127	597.22			

4.4 Implications

Although the subjects were students with minimal systems experience, the experiment is intended to simply test the viability of the framework for selecting components using domain knowledge and generate a quality solution. From that perspective, the experiment demonstrates the overall value of such a system, with one exception (detailed object definition). In fact, if such a knowledge base is used and new developers are able to produce a “quality solution” from these repositories, then an organization might improve its software development productivity. Future

research may also explore the effectiveness of the system when the subjects are both experienced developers, as well as when the developers include a mix of inexperienced and experienced developers, to help better assess the value of the knowledge base and its impact on productivity.

On the technical side, in the current implementation of SADA, the object repository contains common objects found in each of the domains tested. These objects were created using the Java Beans component model, and exhibit predictable behavior with a well-defined object interface. While SADA assumes that such an object repository exists, it is not limited to handling only the Java Beans based

Table 3 Independent samples *T*-test of different scores for with SADA vs. without SADA

Group	<i>N</i>	Mean	Std Dev	Pooled variance estimate				
				<i>F</i>	<i>p</i>	<i>t</i>	df	1-tail <i>p</i>
a. <i>T</i> -test for total score								
With SADA	64	40.2813	3.3769	34.580	0.0000	15.378	126	0.0000
No SADA	64	25.2500	7.0531					
b. <i>T</i> -test for objectives score								
With SADA	64	12.5156	1.6035	17.990	0.000	11.990	126	0.000
No SADA	64	7.5938	2.8659					
c. <i>T</i> -test for processes score								
With SADA	64	8.1563	0.7811	31.622	0.000	10.951	126	0.000
No SADA	64	5.5625	1.7263					
d. <i>T</i> -test for actions score								
With SADA	64	6.4688	0.6416	23.965	0.000	10.629	126	0.000
No SADA	64	4.5625	1.2833					
e. <i>T</i> -test for objects score								
With SADA	64	4.7813	0.5185	59.427	0.000	11.791	126	0.000
No SADA	64	2.5000	1.4584					
f. <i>T</i> -test for consistency score								
With SADA	64	8.3750	0.9554	23.389	0.000	13.721	126	0.000
No SADA	64	5.0313	1.6994					

repository. The only requirement of SADA is that the object repository be populated with components with a well-defined interface and predictable behavior. In the extreme case, the object repository may contain legacy code with appropriate “wrapper” objects that provide the necessary interface. While SADA assumes that the objects in the repository meet certain standards, certifying these objects for quality is beyond the scope of this research.

By allowing developers to access domain knowledge to extract process requirements first and use this knowledge to select components from design repositories, the system reduces design-induced bias (i.e. a pre-disposition to the use of certain system artifacts). Other researchers have used similar approaches to address bias both in the analysis and design phases [84, 85]. However, this approach does not prevent experienced systems designers to select components independent of the domain knowledge repositories.

5 Discussion

The proposed framework uses knowledge about a business domain to help the developer retrieve appropriate software components for systems development. This approach has several benefits. First, explicit representation of the domain knowledge, based on a business domain (e.g. sales, human resources, purchasing, etc.) or industry (e.g. banking, insurance, finance, etc.) [86], is relatively stable. Thus, it

yields more robust requirements for an information system than requirements defined by a user at a given point in time based on a perceived need. In fact, the software productivity consortium (www.software.org) calls for the use of domain concepts, when appropriate, before one begins any system development activity, rather than focus too quickly or narrowly on system design [42].

Second, by starting the development process with an understanding of the business objectives and processes a new system must support, a firm can develop performance metrics for the new system and even benchmark its current processes against those embedded in the knowledge base for possible redesign. Third, such an analysis, independent of design motives, is valuable when more and more firms are intent on defining their process requirements, and want an external vendor to provide the software or services to meet these requirements. In other words, the framework with domain knowledge can be helpful for the process and systems analysis phases, even if the firm chooses not to move to the system design phase.

Finally, as organizations keep demanding faster development of application systems with limited IS resources, the responsibility for building these applications falls not only on limited (and sometimes inexperienced) developer resources, but also on experienced users, who have some domain understanding and technical know-how to initiate the system specification process with relevant components. The framework presented in this

research thus supports the search process for selecting the right components, if one must design a system under these conditions. This should facilitate quicker and easier software development and, in doing so, contribute to the notion of agile computing.

The initial results are encouraging. However, this study has some limitations with the most obvious being the use of student subjects in the empirical validation. Ideally, the experiment should use systems analysts and experienced developers from industry. However, we did not have access to such people. Hence, upper level MIS students were used because of their knowledge and training in systems development. The results indicate that the proposed approach and the system help novice users, although, further empirical validation with experienced analysts is needed. Another limitation is that the study did not take into account the development processes and the tools used in industry. Without having access to the computing resources and the development tools typically available in an organization, it is difficult to replicate the same software development environment in a university setting.

6 Conclusion

A framework for using domain knowledge and software component repositories to select specific software components has been proposed and implemented in a prototype. The system uses both domain model knowledge and object repositories to assist a developer in mapping the functional requirements of the target system into a set of detailed objects, attributes and methods that could be integrated into the target system. This is intended to support a rapid and agile development process. A number of rules for consistency checking are incorporated in the methodology. SADA supports repackaging of objects, but not object customization. The knowledge provided by SADA identifies potential inconsistencies. Future versions may provide guidance on how to resolve them. Furthermore, each customized object may become part of the repository with appropriate validation and indexing for context, so the repositories will evolve over time.

References

1. Y.Y. Yusuf, M. Sarhadi, A. Gunasekaran, *Int. J. Product. Econ.* **62**, 33–43 (1999)
2. M. Lippert, Large Refactorings in Agile Software Development, Workshop on Beyond Green-Field Software Development: Strategies for Reengineering and Evolution, OOPSLA (2003)
3. S. Nerur, R. Mahapatra, G. Mangalraj, *Commun. ACM* **48**, 72–78 (2005)
4. H. Sharp, et al., *Agile Development: Opportunity or Fad?* (Conference on Human Factors in Computing Systems, Montréal, Québec, Canada, 2006) pp. 32–35
5. R.L. Nord, J.E. Tomayko, *IEEE Software* **23**, 47–53 (2006)
6. T. Mens, T. Tourwe, *IEEE T. Software Eng.* **30**, 126–139 (2004)
7. C.J. Neill, B. Gill, *IT Professional Manage.* **5**, 33–38 (2003)
8. P. Vitharana, H. Jain, *Inform. Manage.* **37**, 297–309 (2000)
9. C. Pearson, *Health Manage. Technol.* **20**, 34–35 (1999)
10. N.S. Gill, P.S. Grover, *ACM SIGSOFT Software Eng. Notes* **28**, 30–34 (2003)
11. M. Goulao, *Component-Based Software Engineering: A Quantitative Approach OOPSLA'05* (San Diego, CA, 2005), pp. 238–239
12. D. Kiely, *Informationweek* **677**, 1A–6A (1998)
13. Z. Gu, K.G. Shin, 26th IEEE Int. Real-Time Syst. Symposium 167–168 (2005)
14. R. Due, *Inform. Syst. Manage.* **1**, 92–95 (2000)
15. I. Crnkovic, *Component-based Software Engineering for Embedded Systems International Conference on Software Engineering* (St. Louis, MO, 2005), pp. 712–713
16. A. Patrizio, *Informationweek* **799**, 81–86 (2000)
17. P. Vitharana, F.M. Zahedi, H. Jain, *Commun. ACM* **46**, 97–102 (2003)
18. M. Lindvall et al. *IEEE Computer* **37**, 26–34 (2004)
19. D.J. Reifer, *IEEE Software* **19**, 16–18 (2002)
20. P. Abrahamsson et al., *ICSE* 244–254 (2003)
21. K. Beck, *IEEE Computer* **32**, 70–77 (1999)
22. K. Beck, C. Andres, *Extreme Programming Explained: Embrace Change* (Addison-Wesley, 2004)
23. A. DeHondt, G. Brandyberry, *e-Informatica* **1**, 45–60 (2007)
24. J.A. Highsmith, *Adaptive Software Development: A Collaborative Approach to Managing Complex Systems* (Dorset House Publishing, New York, 2000)
25. S. Bayer, J. Highsmith, *Am. Programmer* **7**, 35–42 (1994)
26. J. Stapleton, *Dynamic Systems Development Method – The Method in Practice* (Addison Wesley, 1997)
27. S.R. Palmer, J.M. Felsing, *A Practical Guide to Feature-Driven Development* (2002)
28. P. Coad, E. LeFebvre, J. De Luca, *Java Modeling In Color With UML: Enterprise Components and Process* (Prentice Hall, 2000)
29. DSDM Consortium, *Dynamic Systems Development Method, version 3* (Ashford, 1997)
30. K. Schwaber, Serum Development Process, Workshop on Business Object Design and Implementation, OOPSLA'95 (1995)
31. K. Schwaber, M. Beedle, *Agile Software Development with Scrum* (Upper Saddle River. Prentice-Hall, NJ, 2002)
32. K.S. Barber, J. Holt, G. Baker, 14th international conference on Software engineering and knowledge engineering SEKE '02, Ischia, Italy, 2002, pp. 225–232
33. J. Meekel et al. *Software Eng. Notes* **22**, 75–80 (1997)
34. W. Tracz, *Software Eng. Notes* **19**, 52–56 (1994)
35. S. Chan, T. Lammers, *Reusing Distributed Object Domain Framework*. 5th Intl.Conf. on Software Reuse (Victoria, BC, 1998), pp. 216–223
36. V. Sugumaran, R. Bose, *J. Intelligent Technol.* **8**, 82–94 (1995)
37. V. Peristeras, K. Tarabanis, *Proceedings of the 6th International Conference on Electronic Commerce ICEC '04* 471–479 (2004)
38. S. Jarzabek, *ACM SIGSOFT Software Eng. Notes* **22**, 65–74 (1997)
39. R.B. France, T.B. Horton, *Applying Domain Analysis and Modeling: An Industrial Experience*. Proceedings of the 1995 Symposium on Software reusability SSR '95, Seattle, WA, 1995, pp. 206–214
40. Y. Choi, M. Heimdahl, *18th IEEE International Conference on Automated Software Engineering* 314 (2003)
41. K.C. Kang, *Proceedings of Fifth International Conference on Software Reuse* 354–355 (1998)
42. W.B. Frakes, K. Kang, *IEEE T. Software Eng.* **7**, 529–536 (2005)
43. M.T. Lubars, M.T. Harandi, *Addressing Software Reuse Through knowledge-Based Design*, by Ted J. Biggerstaff, Alan J. Perlis, *Software Reusability I, II* (1989) 345–377

44. M.A. Simos, *Organization Domain Modeling (ODM): Formalizing the Core Domain Modeling Life Cycle*. Proceedings of the 1995 Symposium on Software reusability SSR '95, Seattle, WA, 1995, pp. 196–205
45. H. Gomaa, L. Kerschberg, G.A. Farrukh, 6th IEEE Int. Conference Eng. Complex Comput. Syst. **11–14**, 50–60 (2000)
46. S. Jarzabek, H. Zhang, *Proceedings of Fifth IEEE International Symposium on Requirements Engineering* **27–31**, 166–173 (2001)
47. T. Ravichandran, IEEE T. Eng. Manage. **52**, 249–268 (2005)
48. A.I. Morch et al. Commun. ACM **47**, 59–62 (2004)
49. A.W. Brown, *Large-scale Component-based Development* (Upper Saddle River, Prentice Hall, NJ, 2000)
50. P. Herzum, O. Sims, *Business Component Factory: A Comprehensive Overview of Component-Based Development for the Enterprise*. (John Wiley, New York, 2000)
51. I. Crnkovic et al. ACM SIGSOFT Software Eng. Notes **30**, 1–9 (2005)
52. P. Vitharana, H. Jain, F. Zahedi, IEEE T Syst. Man Cybernetics **34**, 460–474 (2004)
53. L.F. Capretz, M.A.M. Capretz, D. Li, 27th Annual Conference of the IEEE Industrial Electronics Society **3**, 1834–1837 (2001)
54. A. Andrews, S. Ghosh, E.M. Choi, *International Conference on Software Maintenance* 359–368 (2002)
55. P.T. Cox, B. Song, *IEEE Symposia on Human-Centric Computing Languages and Environments* 304–311 (2001)
56. I. Crnkovic et al. ACM SIGSOFT Software Eng. Notes **29**, 1–7 (2004)
57. N.S. Gill, ACM SIGSOFT Software Eng. Notes **31**, 1–3 (2006)
58. H. Jain, P. Vitharana, F. Zahedi, The DATA BASE Advances Inform. Syst. **34**, 48–63 (2003)
59. H. Yao, L. Etzkorn, *ACMSE'04*. (Huntsville, Alabama, 2004) 110–115
60. A. Mili, R. Mili, R.T. Mittermeir, Annu. Software Eng. **5**, 349–414 (1998)
61. D. Hemer, *Specification-Based Retrieval Strategies for Component Architectures*. Australian Software Engineering Conference 233–242 (2005)
62. Y. Park, P. Bai, *Generating Samples for Component Retrieval by Execution*, Technical Report, University of Windsor (Windsor, Ontario, Canada, 1997)
63. M.A. Barbosa, *A Refinement Calculus for Software Components and Architectures*, *ESEC-FSE'05* (Lisbon, Portugal, 2005), pp. 377–380
64. L. Jilani et al., *Retrieving Software Components That Minimize Adaptation Effort*, In Proc. of the 12th IEEE Intl. Automated Software Engineering Conference, IEEE Computer Society Press (1997), pp. 255–262
65. O.P. Rotaru, M. Dolore, IEEE International Conference on Computer Systems and Applications (2005)
66. S. Faustle, M.G. Fugini, E. Damiani, Software Pract. Exp. **26**, 491–530 (1996)
67. N.S. Gill, ACM SIGSOFT Software Eng. Notes **28**, 4–7 (2003)
68. M.L. Nelson, T. Poulis, ACM OOPS Messenger **6**, 28–36 (1995)
69. J. Cheesman, J. Daniels, *UML Components: A Simple Process for Specifying Component-Based Software* (Addison-Wesley, 2001)
70. D. Hemer, *Proceedings of the Twenty-eighth Australasian conference on Computer Science* 38 (2005)
71. B.H.C. Cheng, J.J. Jeng, IEEE T. Knowledge Data Eng. **9**, 341–349 (1997)
72. J. Penix, *Compositional Specification of Software Architecture*. Proceedings of the Third International Workshop on Software architecture, Orlando, FL, 1998, pp. 113–116
73. A.M. Zaremski, J.M. Wing, ACM T. Software Eng. Methodol. **6**, 333–369 (1997)
74. Y. Chen, B.H.C. Cheng, *A Semantic Foundation for Specification Matching*, in *Foundations of Component-Based Systems*, ed. by M. Sitaraman, G. Leavens (Cambridge University Press, 2000)
75. Y. Chen, B.H.C. Cheng, *Formalizing and Automating Component Reuse*. IEEE Proc. of International Conference on Tools with Artificial Intelligence (1997)
76. J. Penix, P. Alexander, Chapter 10, *Component-Based Software Engineering: Case Studies*, in REBOUND: A Framework for Automated Component Adaptation (World Scientific, 2004)
77. Y. Ding, N. Napier, *Measurement Framework for Assessing Risks in Component-Based Software Development*. Proceedings of the 39th Annual Hawaii International Conference on System Sciences 9 (2006)
78. V. Sugumaran, V. Storey, DATABASE Advances Inform. Syst. **34**, 8–24 (2003)
79. X. Wang, C.W. Chan, H.J. Hamilton, *Design of Knowledge-based Systems with the Ontology-domain-system Approach*. 14th international conference on Software engineering and knowledge engineering SEKE '02 (Ischia, Italy, 2002), pp. 233–236
80. V. Sugumaran, M. Tanniru, V. Storey, Commun. ACM **43**, 312–322 (2000)
81. P. Khambekar et al., *6th Workshop on Information Technologies and Systems* (Cleveland, 1996), pp. 299–303
82. E. Friedman-Hill, *Jess, the Expert System Shell* (Sandia National Laboratories, Livermore, CA, 2007). URL: <http://herzberg.ca.sandia.gov/jess>
83. D. Batra, J.A. Hoffer, R. Bostrom, Commun. ACM **33**, 126–139 (1990)
84. R. Agarwal, M. Tanniru, Knowledge Acquisition **3**, 421–441 (1991)
85. R. Agarwal, L. Roberge, M. Tanniru, Inform. Manage. **27**, 261–274 (1994)
86. J.M. Neighbors, *Proceedings of the Third International Conference on Software Reuse* 6–13 (1994)